Week 10

Back-End Development

Web Applications Development — Undergraduate Programme

Cyber.SoHo Educational Hub | Schiller International University

Today's Roadmap

1. Lesson 1: Node.js — The runtime that lets JavaScript run on servers
2. Lesson 2: Express — The framework that makes building web servers easy
3. Lesson 3: Front-End Frameworks — Vue (carried over from front-end week)
4. We cross from building the "face" (front-end) to building the "brain" (back-end)
5. By the end of this hour: you will understand how these three technologies form a modern full-stack

Front-End vs Back-End

Component	Role	Technologies
Client (Front-End)	User interface & interaction	HTML, CSS, JS, Vue
Server (Back-End)	Logic, data, authentication	Node.js, Express, DBs
Communication	Request ↔ Response cycle	HTTP, JSON, REST

Why Full-Stack JavaScript?

- Before 2009: Front-end team used JavaScript; back-end used Java/Python/PHP/Ruby
- Two languages = translation overhead, slower development, more bugs
- Node.js unified the stack: ONE language (JavaScript) everywhere
- Shared codebases, shared libraries, faster team velocity
- Netflix, LinkedIn, PayPal, NASA, and Uber all adopted Node.js

Lesson 1: Node.js

The Runtime That Freed JavaScript

What Is Node.js?

- Created in 2009 by Ryan Dahl
- A runtime environment that executes JavaScript outside the browser
- Built on Google Chrome's V8 Engine (written in C++)
- Free, open-source, and cross-platform (Windows, macOS, Linux)
- Runtime Environment = the "operating room" — provides everything code needs to execute
- Official site: https://nodejs.org/

The V8 Engine

- V8 is the component inside Google Chrome that compiles JS to machine code
- Written in C++ for raw speed — one of the fastest script engines ever built
- Node.js extracted V8 from the browser and added server capabilities:
- → Read/write files on disk
- → Listen on network ports and handle connections
- → Access databases and external APIs (Application Programming Interfaces)
- → Run as a background process on servers

NPM (Node Package Manager)

- Ships with every Node.js installation — no separate download
- Online catalogue of 2.1+ million free, reusable code packages
- The largest software registry on Earth
- npm init -y → Create a new project (generates package.json)
- npm install express → Add a package to your project
- npm start → Run the project's start script
- Official site: https://www.npmjs.com/

The Event Loop — Coffee Shop Analogy

- Traditional (Blocking): Barista makes one drink at a time, everyone waits
- → 10 drinks × 3 minutes = last customer waits 30 minutes
- Node.js (Non-Blocking): Barista starts drink #1, immediately takes order #2
- → While machine drips coffee #1, barista steams milk for #2
- → When machine beeps (an EVENT), barista grabs finished drink
- → Nobody waits unnecessarily
- This is the Event Loop — Node's beating heart

Event Loop — How It Works

- 1. Request arrives → Event Loop picks it up (single thread)
- 2. Fast task (computation)? → Handle immediately, respond
- 3. Slow task (file I/O, database, network)? → Delegate it and move on
- 4. When the slow task finishes → a CALLBACK function fires
- 5. Event Loop delivers the response to the right client
- Result: One thread handles THOUSANDS of simultaneous connections
- Key concept: Callback = "call me when you are done"

Installing Node.js

1. 1. Visit https://nodejs.org/
2. 2. Download the LTS (Long-Term Support) version — stable and battle-tested
3. 3. Run the installer (follow defaults)
4. 4. Open a terminal and verify: node --version
5. 5. Expected output: v22.x.x (or similar current version)
6. 6. Also verify NPM: npm --version

KEY TAKEAWAY

Node.js

Node.js is NOT a language and NOT a framework — it is a RUNTIME. It frees JavaScript from the browser, letting it run on servers. Its non-blocking Event Loop handles thousands of connections with a single thread, making it ideal for real-time, data-intensive web applications.

Recommended Video: "Node.js in 7 Easy Steps" by Fireship
https://www.youtube.com/watch?v=ENrzD9HAZK4

Lesson 2: Express

The Framework That Tames the Server

What Is Express?

- A minimal, unopinionated web framework for Node.js
- Sits on top of Node's built-in http module — adds structure and convenience
- Handles: routing, middleware, JSON parsing, static file serving
- Maintained by the OpenJS Foundation (https://openjsf.org/)
- Official site: https://expressjs.com/
- Analogy: If Node.js is the engine, Express is the steering wheel & dashboard

Express vs Raw Node.js

Feature	Raw Node.js	Express
Routing	Manual string matching	app.get('/path', handler)
Middleware	Build your own chain	app.use(fn) — plug & play
JSON Parsing	Manual Buffer + JSON.parse	express.json() — one line
Static Files	Read from disk manually	express.static('public')
Ecosystem	Limited integration	Thousands of NPM middleware

Core Concept — Routing

HTTP Method	Purpose	Express Syntax
GET	Retrieve / Read data	app.get('/students', fn)
POST	Create new data	app.post('/students', fn)
PUT	Update (full replace)	app.put('/students/:id', fn)
PATCH	Update (partial)	app.patch('/students/:id', fn)
DELETE	Remove data	app.delete('/students/:id', fn)

Core Concept — Middleware

- Middleware = functions that execute during the request-response cycle
- Airport Security Analogy: luggage (request) passes through checkpoints
- Each checkpoint can: inspect, modify, reject, or pass the request along
- Pipeline: Request → Logger → Auth Check → JSON Parser → Route Handler → Response
- Every middleware receives: (req, res, next)
- Calling next() passes control to the next function in the chain
- Forgetting next() causes the request to hang forever!

Core Concept — REST

- REST = Representational State Transfer
- An architectural style for designing networked applications
- Uses standard HTTP methods (GET, POST, PUT, DELETE) on resources
- Resources identified by URLs: /students, /students/42
- Responses typically in JSON (JavaScript Object Notation) format
- Stateless: each request contains all info needed — no server-side sessions
- The standard approach for building modern web APIs

Installing Express

- Step 1: Create project folder → mkdir my-express-app && cd my-express-app
- Step 2: Initialise NPM → npm init -y (creates package.json)
- Step 3: Install Express → npm install express
- package.json = your project's ID card (name, version, dependencies)
- node_modules/ = folder where NPM stores downloaded packages
- Now you can require('express') in any .js file in the project

KEY TAKEAWAY

Express

Express is the most popular Node.js framework because it does exactly what you need and nothing more. Master two concepts — middleware pipelines and routing — and you can build any back-end. Its declarative syntax (app.get, app.post) replaces hundreds of lines of raw Node.js code.

Recommended Video: "RESTful APIs in 100 Seconds" by Fireship
https://www.youtube.com/watch?v=-MTSQjw5DrM

Lesson 3: Front-End Frameworks — Vue

The Progressive Framework for Reactive UIs

What Is Vue?

- Pronounced "view" — like looking out a window
- A progressive JavaScript framework for building user interfaces
- Created in 2014 by Evan You (former Google engineer)
- "Progressive" = adopt incrementally — from one widget to a full SPA
- SPA = Single-Page Application — loads once, updates dynamically
- Used by: Alibaba, GitLab, Nintendo eShop, Xiaomi, Grammarly
- Official site: https://vuejs.org/

Vue vs React vs Angular

Aspect	Vue	React	Angular
Type	Progressive framework	UI library	Full framework
Templates	HTML-based	JSX (JS+HTML)	TypeScript + HTML
Learning Curve	Gentle	Moderate	Steep
Data Binding	Two-way (v-model)	One-way (state→view)	Two-way (ngModel)
Creator	Evan You	Meta (Facebook)	Google
First Release	2014	2013	2010

Core Concept — Components

- A component = a self-contained, reusable UI building block
- A nav bar, a login form, a single tweet — each is a component
- Vue uses SFC (Single-File Components) with .vue extension
- Each SFC has three sections:
- 1. <template> — HTML structure (what it looks like)
- 2. <script> — JavaScript logic (what it does)
- 3. <style> — CSS styling (how it is styled)
- Everything for one UI piece lives together in one file

Core Concept — Reactivity

- Reactivity = the UI automatically updates when data changes
- Step 1: Declare reactive data → const count = ref(0)
- Step 2: Use in template → {{ count }}
- Step 3: Change data → count.value++
- Step 4: Vue detects the change → re-renders ONLY affected DOM elements
- You NEVER manually call document.getElementById() or innerHTML
- Vue's virtual DOM diffing ensures maximum efficiency

Core Concept — Vue Directives

Directive	Purpose	Example
v-if	Conditional rendering	Visible
v-for	Loop through array	<li v-for="item in items">
v-model	Two-way data binding	<input v-model="name">
v-bind / :	Dynamic attributes	<img :src="url">
v-on / @	Event handling	<button @click="save">
v-show	Toggle CSS display	

Two-Way Data Binding with v-model

- v-model connects a form input to a JavaScript variable
- User types → variable updates instantly
- Code changes variable → input field updates instantly
- No manual event listeners needed!
- Traditional JS: addEventListener → get value → update variable → update DOM
- Vue: v-model="username" — done.
- This is what makes Vue feel "magical" to beginners

The Vue Reactivity Cycle

1. 1. User interacts (types, clicks, submits)
2. 2. Data updates via v-model or event handlers
3. 3. Vue detects change through reactive proxy system
4. 4. Virtual DOM diffs — compares old state vs new state
5. 5. Only changed elements re-render — efficient and fast
6. 6. User sees updated UI → triggers new actions → cycle repeats
7. This cycle runs continuously as long as the app is open

Getting Started with Vue (Zero Build Tools)

- Load Vue from CDN: <script src="https://unpkg.com/vue@3/dist/vue.global.js">
- Create a mount point:
- Use the Composition API with setup() function
- ref() makes values reactive — Vue tracks all changes
- No terminal, no npm, no build tools needed for learning!
- Just save an .html file and open it in any browser
- Perfect for classroom demonstrations and quick experiments

KEY TAKEAWAY

Vue

Vue is the approachable front-end framework that plays beautifully with any back-end. Its reactivity system lets you describe WHAT the UI should look like for a given state, and Vue figures out HOW to update the DOM efficiently. Combined with Node.js and Express, Vue completes a full-stack JavaScript environment.

Recommended Video: "Vue.js Explained in 100 Seconds" by Fireship
https://www.youtube.com/watch?v=nhBVL41-_Cw

Wrap-Up

Putting It All Together

The Full-Stack JavaScript Picture

Layer	Technology	Role
Front-End	Vue.js	Reactive UI, components, SPA
Back-End Runtime	Node.js	JS outside browser, Event Loop
Back-End Framework	Express	Routing, middleware, REST APIs
Communication	HTTP + JSON	Client ↔ Server data exchange

Three Things to Remember

- 1. Node.js = RUNTIME (not a language, not a framework)
- Non-blocking Event Loop handles thousands of connections
- 2. Express = MINIMAL FRAMEWORK
- Middleware + Routing = any back-end you can imagine
- 3. Vue = PROGRESSIVE FRAMEWORK
- Reactivity + Components = dynamic, efficient UIs
- Together: full-stack JavaScript — one language, front to back

Abbreviations Reference (1/2)

Abbreviation	Full Form
API	Application Programming Interface
CDN	Content Delivery Network
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
I/O	Input/Output
JSON	JavaScript Object Notation

Abbreviations Reference (2/2)

Abbreviation	Full Form
LTS	Long-Term Support
NPM	Node Package Manager
PHP	PHP Hypertext Preprocessor
REST	Representational State Transfer
SFC	Single-File Component
SPA	Single-Page Application
UI	User Interface
URL	Uniform Resource Locator

Essential Links

Resource	URL
Node.js	https://nodejs.org/
NPM	https://www.npmjs.com/
Express.js	https://expressjs.com/
Vue.js	https://vuejs.org/
MDN Web Docs	https://developer.mozilla.org/
freeCodeCamp	https://www.freecodecamp.org/
VirusTotal	https://www.virustotal.com/gui/home/url

Disclaimer

- IP: All material is the intellectual property of Cyber.SoHo Educational Hub
- Copyright: Not affiliated with, endorsed by, or sponsored by any referenced companies
- External Links: Verify safety at https://www.virustotal.com/gui/home/url
- Liability: Provided "as is" for educational purposes only
- Ethics: Apply all skills responsibly, legally, and ethically

Thank You!

Cyber.SoHo Educational Hub | Schiller International University

"From the Industry to the Classroom — Building the IT's Future, Today and Together!"